Methods: Network analysis of hypertension and ovarian cancer

October 20, 2026

1 Human interactome

We used the human protein–protein interactome compiled by Menche et al. (2015), which contains 141,296 physical interactions among 13,460 proteins. The data come from binary yeast two-hybrid screens, literature-curated interactions, protein complexes, kinase–substrate pairs, signaling, metabolic enzyme-coupled and regulatory interactions. In this network genes are identified by NCBI Entrez Gene IDs. We mapped each ID to its approved gene symbol using the `entrez_id` field of the HGNC complete gene set. Of the 13,460 Entrez IDs, 13,334 (99.1%) received a symbol.

The remaining 126 IDs had no HGNC symbol. We queried NCBI Gene (E-utilities esummary) for their status: 74 had been replaced by a newer ID, 31 had been discontinued, and 21 were live records with no HGNC entry. To check the mapping, we confirmed that five well-established interacting pairs were present in the mapped network (TP53–MDM2, BRCA1–BARD1, BRCA2–PALB2, ESR1–NCOA3, CDK4–CCND1).

We then cleaned the network in three steps:

1. We removed self-interactions, leaving 138,427 of the original 141,296 edges.
2. We treated edges as undirected and merged duplicate pairs (A–B and B–A). This step found no duplicates.
3. We removed the 564 edges (0.41%) in which at least one gene had no symbol.

The final network contains 13,270 genes and 137,863 interactions. All downstream analyses use this network, modelled as an unweighted, undirected graph.

2 Disease gene sets

We took gene–disease associations from the Open Targets Platform for three conditions:

- ovarian cancer (OC, MONDO_0008170)
- hypertensive disorder (HTN, MONDO_0005044)
- systolic blood pressure (SBP, EFO_0006335)

SBP is closely related to HTN, so we included it as a positive control.

Open Targets gives each gene an overall association score (`globalScore`, 0–1). This score combines evidence from genetics, somatic mutations, literature, animal models and other sources. For each disease, we kept genes with a score of at least 0.1. We chose this low cutoff to include as many

associated genes as possible, while still removing very weakly associated genes and keeping the computation manageable. We then kept only those genes that are present in the interactome, because genes with no known interactions cannot be placed on the network. To check that the results do not depend on the score cutoff, we repeated the overlap analysis at cutoffs of 0.2, 0.3, 0.4 and 0.5.

Disease	Genes in Open Targets	Score ≥ 0.1	In interactome
OC	15,834	1,555	1,428
HTN	10,012	2,402	1,830
SBP	4,884	2,877	2,158

Table 1: Disease gene sets.

3 Gene overlap

For each pair of diseases A and B, we counted the shared genes $|A \cap B|$ and computed three measures: the Jaccard index, the overlap coefficient, and fold enrichment over random expectation.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}, \qquad O(A,B) = \frac{|A \cap B|}{\min(|A|,|B|)}, \qquad \text{fold} = \frac{|A \cap B|}{|A|\,|B|/N} \tag{1}$$

Here $N = 13{,}270$ is the number of genes in the interactome. We used all interactome genes as the background.

We tested whether the overlap is larger than chance with a one-sided hypergeometric test. This test gives the probability of seeing at least $k = |A \cap B|$ shared genes when $|B|$ genes are drawn at random from N genes, of which $|A|$ belong to disease A:

$$P(X \geq k) = \sum_{i=k}^{\min(|A|,|B|)} \frac{\binom{|A|}{i}\binom{N-|A|}{|B|-i}}{\binom{N}{|B|}} \tag{2}$$

3.1 Numerical evaluation

To avoid numerical overflow, we evaluated the binomial coefficients in log space with the log-gamma function.

Overflow. Each term contains three binomial coefficients, $\binom{a}{b} = a!/\big(b!\,(a-b)!\big)$, and these become extremely large for gene sets of this size. For example, $\binom{13{,}270}{1{,}830}$ is the number of ways to choose the 1,830 HTN genes from the network, and it is about 10^{2310}. The largest 64-bit floating-point number is about 10^{308}. A direct calculation therefore overflows, even though the final probability is an ordinary number.

Log space. We therefore worked with logarithms. Because $\ln(xy) = \ln x + \ln y$ and $\ln(x/y) = \ln x - \ln y$, each term becomes a sum and difference of log-binomial coefficients:

$$\ln P(X = i) = \ln \binom{|A|}{i} + \ln \binom{N-|A|}{|B|-i} - \ln \binom{N}{|B|} \tag{3}$$

In log space, the same coefficient is only $\ln \binom{13{,}270}{1{,}830} \approx 5{,}319$, which is easily stored. The probability is recovered at the end as $P(X = i) = \exp[\ln P(X = i)]$. Because a probability is at most 1, its logarithm is at most 0, so this step cannot overflow.

Log-gamma function. The log-binomial coefficients must themselves be computed without first forming $\binom{a}{b}$ or the factorials, since 13,270! alone has about 48,950 digits. For non-negative integers n, the gamma function satisfies $\Gamma(n+1) = n!$, so $\ln n! = \ln \Gamma(n+1)$ and

$$\ln \binom{a}{b} = \ln \Gamma(a+1) - \ln \Gamma(b+1) - \ln \Gamma(a-b+1) \tag{4}$$

4 Network localization

We asked whether each disease's genes cluster together in the interactome, forming a disease module. Following Menche et al. (2015), we used two measures.

Largest connected component (S). We built the subgraph induced by a disease's genes, keeping only edges between two disease genes, and took the size of its largest connected component.

Mean shortest distance (d_s). For each disease gene, we found the shortest-path distance to the nearest other gene of the same disease, then averaged over all genes. Shortest paths were computed in the full interactome:

$$\langle d_s \rangle = \frac{1}{|A|} \sum_{a \in A} \min_{a' \in A,\, a' \neq a} d(a, a') \tag{5}$$

Here $d(a, a')$ is the number of edges on the shortest path between the two genes. Genes with no path to any other disease gene were left out of the average.

A disease forms a module when S is larger than expected for random gene sets of the same size and d_s is smaller.

5 Network separation

To measure how close two disease modules sit in the interactome, we used the network separation of Menche et al. (2015):

$$s_{AB} = \langle d_{AB} \rangle - \frac{\langle d_{AA} \rangle + \langle d_{BB} \rangle}{2} \tag{6}$$

Here $\langle d_{AA} \rangle$ and $\langle d_{BB} \rangle$ are the mean shortest distances within each disease, defined as d_s above.

$\langle d_{AB} \rangle$ is the mean distance between the two diseases:

$$\langle d_{AB} \rangle = \frac{1}{|A| + |B|} \left[\sum_{a \in A} \min_{b \in B} d(a, b) + \sum_{b \in B} \min_{a \in A} d(a, b) \right] \tag{7}$$

The first sum runs over the genes of A, each with its distance to the nearest gene of B. The second sum runs over the genes of B, each with its distance to the nearest gene of A. A gene shared by both diseases has $d(a, a) = 0$.

A negative s_{AB} means the two disease modules overlap in the network, and a positive value means they are separated.

6 Statistical significance

We tested S, d_s and s_{AB} against a null model of random gene sets, using uniform randomization. This corresponds to the "full randomization" of Menche et al. (2015).

In this model, a random gene set is drawn by picking genes from the 13,270 network genes uniformly at random and without replacement. Every gene has the same chance of being picked, and no gene appears twice in one set. The random set has the same size as the disease gene set it stands in for, but carries nothing else from the disease genes: neither their number of interactions (degree) nor their positions in the network.

- **Localization (S and d_s):** for a disease with n genes, each repeat draws one random set of n genes.
- **Separation (s_{AB}):** each repeat draws two independent random sets, of $|A|$ and $|B|$ genes. Because the two sets are drawn independently, they share some genes by chance, on average $|A| \cdot |B|/N$.

We drew 10,000 random sets, or pairs of sets, for each disease and each pair, and computed the same measure on each. Together these values give the distribution expected if the disease genes were scattered at random across the network.

For each observed value x, we computed a z-score against the random values, where μ and σ are their mean and standard deviation:

$$z = \frac{x - \mu_{\text{rand}}}{\sigma_{\text{rand}}} \tag{8}$$

We also computed a one-sided empirical p-value, where r is the number of random values at least as extreme as x and $n = 10{,}000$:

$$p = \frac{r + 1}{n + 1} \tag{9}$$

The direction of "extreme" depends on the measure. For S, larger values count, so $z > 1.6$ is significant. For d_s and s_{AB}, smaller values count, so $z < -1.6$ is significant. In both cases this corresponds to $p < 0.05$.

The smallest p-value that 10,000 randomizations can give is about 1×10^{-4}.

One caveat applies to s_{AB}. Shared genes have distance 0. Random sets of the same sizes share only $|A| \cdot |B|/N$ genes on average (197 for OC–HTN, against 383 observed). As a result, part of a negative separation z-score reflects the excess direct gene overlap (see Menche et al. 2015, Supplementary Section 2.2.1).

7 Network visualization

To inspect how the disease genes are arranged in the interactome, we built interactive network views of the disease-gene subnetworks. These views are descriptive and were not used for any statistical test.

Subnetworks. For each view, we took the disease genes (score ≥ 0.1, present in the interactome) and kept only the interactions between them. For OC and HTN, this subnetwork contains 2,875 genes and 16,810 interactions:

- 1,045 genes in OC only

- 1,447 genes in HTN only
- 383 genes shared by both

360 of these genes have no interaction with another disease gene and were shown separately. We also counted the interactions by the groups of the two genes they connect, for example OC only to HTN only (3,864 interactions).

From this subnetwork we derived two focused views:

- **Shared genes.** This view contains the 383 shared genes together with every OC or HTN gene directly linked to one of them, and all interactions among these genes. It shows how the shared genes act as bridges that connect the two diseases.
- **Cross links.** This view keeps only the interactions that connect the two diseases: OC only to HTN only, either group to a shared gene, and shared gene to shared gene. Interactions that stay within one disease (OC only to OC only, HTN only to HTN only) are removed, together with genes left without any interaction.

View	Content	Genes	Interactions	Modules
OC + HTN, all genes	All OC and HTN genes	2,875	16,810	12
OC + HTN, shared genes	Shared genes and their directly linked OC or HTN genes	1,735	14,175	9
OC + HTN, cross links	Only interactions between the two diseases, or involving a shared gene	2,260	10,353	11
OC + HTN + SBP, all genes	All genes of the three diseases	3,991	23,422	13
OC + HTN + SBP, shared genes	Genes in two or more diseases and their directly linked disease genes	3,109	22,459	16
RAS	Renin–angiotensin system genes and their directly linked disease genes	126	123	–

Table 2: Network views.

Renin–angiotensin system (RAS). We selected 23 genes of the RAS. 21 of them are in the interactome; CPA3 and MRGPRD are not. The RAS view shows these 21 genes together with the 105 OC, HTN or SBP genes directly linked to them. We kept only interactions that involve at least one RAS gene, 12 of which connect two RAS genes. For each RAS gene, we also recorded its disease memberships, its number of links to other RAS genes and to disease genes, and its degree in the full interactome.

Module layout. Most genes in each view form one large connected component, which is too dense to read as a single cloud. We therefore split it into modules with the Louvain community detection method (Blondel et al. 2008).

Louvain divides a network into modules, groups of genes that interact much more with one another than with the rest of the network. It chooses the division that maximises modularity, which compares the number of interactions inside modules with the number expected if the same genes were linked at random.

We chose Louvain for three reasons. First, it is fast enough for networks of thousands of genes and tens of thousands of interactions, such as our views. Second, it finds the number of modules by itself, so we did not have to fix it in advance. Third, it is widely used on biological networks, where densely linked groups of genes often share a function or pathway. This makes the modules a natural way to organise the views. The modules serve only to lay out the views and were not used in any statistical test.

The result of Louvain can change from run to run, so we used the NetworkX implementation with a fixed seed and sorted input. The modules are therefore the same on every run.

The modules were then arranged as follows:

- Modules of at least 20 genes were drawn in their own circle, named after their three most connected genes.
- Smaller modules and the small separate components were pooled into one shared area.
- Genes with no interaction in the view were placed in a grid.

Within each area, gene positions were computed with the ForceAtlas2 force-directed layout (200 iterations), then scaled so that genes keep a similar spacing across modules. The areas were then packed in rows, largest first. The module of every gene was saved as a table next to each view. The small RAS view was laid out instead with a force-directed (Barnes–Hut) simulation that is frozen once it settles.

Visual encoding.

- **Colour:** each disease has its own colour. A gene in several diseases is drawn as a pie with one slice per disease, so the three colours cover all seven possible groups. RAS genes have an additional dark ring.
- **Size:** node size increases with the square root of the gene's number of interactions within the view.
- **Labels:** genes shared by two or more diseases are labelled by name.
- **Interactions between modules:** these are hidden by default, so the structure within modules stays visible. They can be shown all together or for one module at a time.

The views were written as standalone HTML pages using the vis-network library (version 9.1.9). In these pages, hovering over a gene shows its disease scores, its degree in the view and in the full interactome, and its module. Groups of genes can be hidden or shown from the legend.

8 Implementation

8.1 Shortest-path distances

The measures d_s, d_{AB} and s_{AB} all use shortest-path distances in the interactome. The network is unweighted, so every edge counts as one step. On such a graph, breadth-first search (BFS) gives exact shortest distances, and we used it to compute them.

BFS from a source gene proceeds as follows:

1. Give the source distance 0 and place it in a first-in, first-out queue.

2. Remove the gene at the front of the queue.

3. Give each of its neighbours that has not yet been visited a distance one greater than the current gene's, and add it to the back of the queue.

4. Repeat steps 2 and 3 until the queue is empty.

The search reaches every gene at distance 1 before any gene at distance 2, and so on. The first distance assigned to a gene is therefore its shortest distance, and it never needs updating. Genes that are never reached lie in a different connected component, and we marked them as unreachable. One search takes time proportional to the number of genes plus the number of edges.

We ran BFS once from each of the 13,270 genes and stored the results in a 13,270 $\times$ 13,270 matrix D, where $D[i, j]$ is the distance between genes i and j. Each entry was stored as an 8-bit integer (176 MB in total). The largest finite distance in the network is 13, and the value 255 marked unreachable pairs.

All distance measures were then read from D:

- d_s**:** we took the submatrix of D for the disease genes, ignored its diagonal (each gene's distance to itself), and took the minimum of each row.

- d_{AB}**:** we took the submatrix with A genes as rows and B genes as columns. The minimum of each row gives the distance from each A gene to the nearest B gene, and the minimum of each column gives the distance from each B gene to the nearest A gene. A gene in both sets has distance 0 to itself, so shared genes count as 0.

In both cases, unreachable pairs were left out of the averages.

Because the distances are computed only once, each of the 10,000 random repeats needs only lookups in D rather than new graph searches. The distances are identical to those from the pairwise shortest-path computation in the original code of Menche et al.

The breadth-first searches and the random repeats ran in parallel across CPU cores. We split the 10,000 repeats into 100 chunks, each with a fixed seed derived from a base seed of 42, so the results do not depend on the number of cores used.